

Question 1: Backtracking Search

To find a valid shift assignment for the three doctors (D_1, D_2, D_3) across the three shifts (Morning, Afternoon, Night), we can formulate this as a constraint satisfaction problem and solve it using Backtracking Search.

1. Problem Definition & Constraints

- **Variables (Shifts):** $S = \{\text{Morning}, \text{Afternoon}, \text{Night}\}$
- **Domain (Doctors):** $D = \{D_1, D_2, D_3\}$ (each assigned to exactly one shift)
- **Constraints:**
 1. D_1 cannot work the Night shift.
 2. D_2 must work in an earlier shift than D_3 ($\text{Shift}(D_2) < \text{Shift}(D_3)$).
 3. Only one doctor per shift (all assignments must be unique).
 4. D_3 cannot work the Morning shift.
 5. All doctors must be assigned exactly one shift.

2. Backtracking Search Steps

We will assign doctors sequentially to the shifts in chronological order: Morning $\rightarrow$ Afternoon $\rightarrow$ Night.

Step 1: Assign a doctor to the Morning Shift

- Choice 1: Assign D_1 to Morning.
- Constraint Check: D_1 is allowed in the Morning. (Valid)
- Current State: Morning = D_1
- Proceed to Afternoon.

Step 2: Assign a doctor to the Afternoon Shift

- Remaining available doctors: $\{D_2, D_3\}$
- Choice 2.1: Assign D_2 to Afternoon.
 - Constraint Check:
 - Unique assignment check: Valid.
 - Temporal constraint check (D_2 before D_3): D_2 is in the Afternoon, meaning D_3 must be in the Night. This places D_2 before D_3 (Afternoon $<$ Night). (Valid)
 - Current State: Morning = D_1 , Afternoon = D_2

- Proceed to Night.

Step 3: Assign a doctor to the Night Shift

- Remaining available doctor: $\{D_3\}$
- **Choice 3.1:** Assign D_3 to **Night**.
 - **Constraint Check:**
 - Unique assignment check: Valid.
 - D_3 cannot work Morning: D_3 is in the Night. (Valid)
 - D_1 cannot work Night: D_1 is in the Morning, so D_3 is in the Night. (Valid)
 - **Current State:** Morning = D_1 , Afternoon = D_2 , Night = D_3

3. Final Valid Schedule

Since all variables are successfully assigned and all constraints are satisfied without needing to backtrack further, the valid schedule is:

Shift	Assigned Doctor
Morning	D1
Afternoon	D2
Evening	D3

Question 2: BFS Search

The paper mentions a 5×5 grid, but the obstacle positions, Start (S), and Goal (G) are not provided. Without the grid, the exact BFS node expansion, queue updates, optimal path, and cost cannot be determined.

1. Grid Environment Setup (5×5)

Let the grid be indexed using coordinate pairs (r, c) where r is the row (0 to 4) and c is the column (0 to 4).

- **Start (S):** (0,0)
- **Goal (G):** (4,4)
- **Obstacles (X):** Let's assume standard representative obstacles at positions $\{(1,2), (2,2), (3,2)\}$ forming a partial wall, or assume an open grid if unspecified. Let's place an obstacle at (2,1) and (2,3) to make it interesting, or assume standard clear path. Assuming a standard clear grid or simple obstacles to show full expansion: Let's assume obstacles at (1,1) **and** (2,1) to demonstrate path deflection.

2. BFS Step-by-Step Node Expansion & Queue Updates

- **Queue Structure:** Stores paths or nodes to visit in FIFO (First-In, First-Out) order.
- **Allowed Movements:** Up $(-1,0)$, Down $(1,0)$, Left $(0,-1)$, Right $(0,1)$.

Initialization:

- **Queue:** $[(0,0)]$
- **Visited Set:** $\{(0,0)\}$
- **Parent Map:** None

Step 1:

- **Dequeue:** $(0,0)$
- **Check Goal:** $(0,0) \neq (4,4)$
- **Expand Neighbors:**
 - $(0,1)$ [Right] - Valid
 - $(1,0)$ [Down] - Valid
 - Up/Left are out of bounds.
- **Queue Update:** $[(0,1), (1,0)]$
- **Visited:** $\{(0,0), (0,1), (1,0)\}$

Step 2:

- **Dequeue:** $(0,1)$
- **Expand Neighbors:**
 - $(0,2)$ [Right] - Valid
 - $(1,1)$ [Down] - **Obstacle** (Skipped)
 - $(0,0)$ [Left] - Already visited
- **Queue Update:** $[(1,0), (0,2)]$
- **Visited:** $\{(0,0), (0,1), (1,0), (0,2)\}$

Step 3:

- **Dequeue:** $(1,0)$
- **Expand Neighbors:**
 - $(2,0)$ [Down] - Valid
 - $(1,1)$ [Right] - **Obstacle** (Skipped)
- **Queue Update:** $[(0,2), (2,0)]$

- **Visited:** $\{(0,0), (0,1), (1,0), (0,2), (2,0)\}$

(Continuing level-by-level BFS expansion outward around obstacles until reaching (4,4))

3. Optimal Path and Cost

By tracing back the shortest unblocked route from (0,0) to (4,4) avoiding obstacles:

- **Optimal Path Coordinates:**

$(0,0) \rightarrow (0,1) \rightarrow (0,2) \rightarrow (0,3) \rightarrow (1,3) \rightarrow (2,3) \rightarrow (3,3) \rightarrow (3,4) \rightarrow (4,4)$ (or equivalent shortest grid traversal).

- **Total Cost: 8 moves** (each move has a cost of 1).

Question 3: Rescue Robot

a) Constraint Analysis and Impact on Decision-Making

- **Partially Observable Environment & Limited Sensing:** Because the robot can only observe adjacent cells, it lacks a complete global map at the start. It cannot pre-compute a static optimal path. Decision-making must be reactive and dynamic, requiring the robot to update its local map and beliefs as it discovers obstacles or open rooms during traversal.
- **Blocked Cells (Obstacles):** Hard constraints that completely prune certain paths from the search tree, forcing local rerouting when unexpected blockages are encountered.
- **Variable Movement Costs (Standard Cost = 1, Risky Zones = +2):** Modifies the edge weights in the graph search. The robot cannot rely solely on shortest-path distance (like standard BFS); it must weigh the trade-off between a longer safe path versus a shorter path that incurs heavy penalty costs.
- **Cost Minimization & Safe Navigation:** Requires an optimization algorithm that balances cost efficiency with risk aversion.
- **Dynamic Knowledge Update (Online Search):** The agent must interleave searching and execution. It moves to an adjacent observed state, updates its heuristic/cost estimates, and re-plans from its current actual location.

b) Solution Approach: Online A* / Real-Time Adaptive A* (RTAA*)

To handle the partially observable online search scenario with variable edge costs, an Online Search Agent utilizing Real-Time Adaptive A* (RTAA*) or Learning Real-Time A* (LRTA*) is appropriate.

Step-by-Step Execution:

1. **Initialization:** The robot starts at S with initial local beliefs. It sets its heuristic $h(s)$ for all known states using the Manhattan distance to the goal G .

2. **Local Sensing:** At its current state s_{current} , the robot scans all adjacent cells to identify open paths, obstacles, and risky zones. It updates its internal graph representation.
3. **Local Search / Planning:** The robot runs a bounded lookahead search (or A* on its known subgraph) from s_{current} using the evaluation function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the actual path cost incurred from s_{current} to n (incorporating standard moves worth 1 and risky zones worth 3), and $h(n)$ is the estimated cost-to-go.

4. **Action Selection:** The robot selects the best immediate action leading to the neighbor that minimizes $f(n)$.
5. **Execution & Learning:** The robot executes the single physical move to the next state s_{next} . It then updates its heuristic values based on the search experience to improve future decision accuracy (learning from dead ends or unexpected obstacles).
6. **Termination:** Repeat steps 2–5 until the robot reaches the survivor at G .

c) Application of Core AI Concepts

- **Intelligent Agents:** The rescue robot functions as an autonomous agent. It perceives its environment via sensors (checking adjacent cells), processes the data through an internal state representation and cost-evaluation algorithm, and actuates effectors (motors for movement) to achieve its goal.
- **Rationality:** The robot is rational because it acts to maximize its performance measure—specifically, reaching the survivor safely while minimizing cumulative travel and risk penalties based on the evidence available to it at any given moment.
- **Environment Type:**
 - **Partially Observable:** The robot cannot see beyond adjacent cells.
 - **Stochastic / Dynamic:** Obstacles or conditions in unobserved areas may change, or new obstacles are discovered upon sensing.
 - **Sequential:** Current actions directly impact future choices and accumulated costs.
 - **Discrete:** The building layout is represented as a grid of distinct rooms.

d) Justification of the Chosen Approach

An **Online Search Agent utilizing Real-Time A*** is the most suitable choice for this scenario because:

1. **Adapts to Partial Observability:** Offline algorithms (like standard A* or Dijkstra) require complete prior knowledge of the map. Online search explicitly accommodates environments where the map is revealed incrementally.

2. **Handles Variable Costs Effectively:** By factoring edge weights into the $g(n)$ cost function during local lookahead, the agent dynamically avoids high-cost risky zones unless bypassing them creates an prohibitively expensive detour.
3. **Real-Time Responsiveness:** Disaster scenarios demand timely decisions. Bounded lookahead ensures the robot does not stall waiting to compute a global path through an unknown building, allowing it to navigate safely step-by-step.